NumPy and SciPy

NumPy

Construct and Access Arrays:

```
A = column_stack( (u,v) )
A = row_stack( (u,v) )
M = hstack( (A,B) )
M = vstack( (A,B) )
```

Neat trick:

```
a = np.r_[3,[0]*5,-1:1:10j]
```

Condition number: $\mathbf{cond}(A) := \|A\| \, \|A^{-1}\|$

```
np.linalg.cond(A)
```

Rank:

```
np.linalg.matrix_rank(A)
```

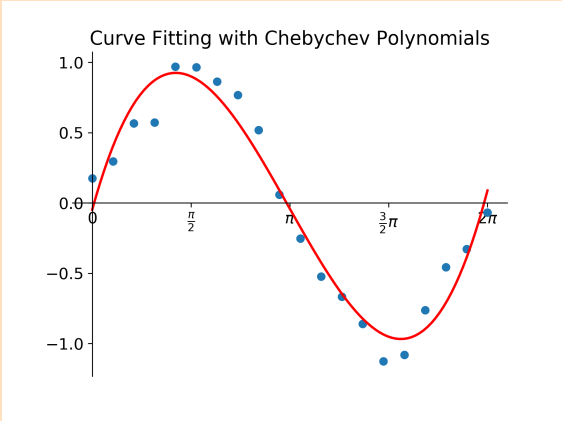
Matrix functions:

```
np.linalg.matrix_power(A, n)     # A^n
linalg.expm(A)                   # matrix exponential
```

Vandermonde-Matrix:

$$V = \mathbf{vander}(\vec{x}) = \begin{pmatrix} x_1^N & \cdots & x_1^2 & x_1 & 1 \\ \vdots & \ddots & \vdots & \vdots & \vdots \\ x_n^N & \cdots & x_n^2 & x_n & 1 \end{pmatrix}$$

```
V = np.vander(x, N)
```



np.polynomial

```
from numpy.polynomial import Polynomial as P
```

$$p(x) = \sum_{k=1}^{n} a_k x^k$$

```
p = P( [a_0, a_1, ..., a_n] )   # Definition of p
p = P.fromroots([1, 2, 3])      # Construct from roots
```

Specify domain and window:

```
p = P( array, domain=[0, 1], window=[-1, 1] )
p.domain = np.array( [a, b] )
p.window = np.array( [a, b] )
```

Integral and Derivative:

```
p.integ()
p.deriv()
```

Polynomial Curve fitting:

```
p = P.fit(x, y, n)
```

SciPy.linalg

```
from scipy import linalg
```

Norm of a vector or matrix:

$$\|A\|_2 = \sqrt{\rho(A^* A)}, \qquad \|A\|_F = \sum_{i,j} |a_{ij}|$$

```
linalg.norm(A)                  # Euclidean/Frobenius
linalg.norm(v, ord=np.inf)      # row-sum norm
```

Moore-Penrose pseudo-inverse:

$$A^\dagger = (A^* A)^{-1} A^*$$

```
linalg.piv(A)
```

Solve Linear System of Equations $A\vec{x} = \vec{b}$

```
linalg.inv(A)          # A^(-1)
x = linalg.solve(A, b)
```

Special function for triangular matrices:

```
x = linalg.solve_triangular(A, b, lower=False)
```

Determinant:

$$|A| = \sum_j (-1)^{i+j} a_{ij} \, |A_{ij}|$$

```
linalg.det(A)
```

Eigenvectors and Eigenvalues:

$$A\vec{v} = \lambda\vec{v}$$

```
lamda = linalg.eigvals(A)       # just eigenvalues

lamda, v = linalg.eig(A)
l1, l2 = lamda            # Eigenvalues
v1 = v[:,0]               # 1st Eigenvector
```

Special functions to calculate Eigenvalues:

`linalg.eigh(A)`	hermitian matrix
`linalg.eigvalsh(A)`	
`linalg.eigh_tridiagonal(A)`	tridiagonal matrix
`linalg.eigvalsh_tridiagonal(A)`	

Schur-Decomposition:

$\forall A \in \mathbb{R}^{n\times n}$, $\sigma(A) \subseteq \mathbb{R} \, \exists$ **orthogonal** U such that:

$$U^* A U = R$$

```
R, U = linalg.schur(A)
```

Singular-Value-Decomposition:

$A \in \mathbb{C}^{n\times n}$ hermitic $\implies \sigma(A) \subseteq \mathbb{R}$

It follows from Schur that:

$$U^* A U = \mathbf{diag}\{\lambda_1, \ldots, \lambda_n\}$$

Applied to $A = M^* M$ we have (if M regular):

$$M = U \Sigma V^{\mathrm{H}}, \quad \Sigma = \mathbf{diag}\{\sigma_1, \ldots, \sigma_n\} > 0$$

```
M, N = A.shape
U, s, Vh = linalg.svd(A)
Sig = linalg.diagsvd(s, M, N)
U @ Sig @ Vh
```

LU- and QR-Decomposition:

$$A = PLU, \quad A = QR$$

```
P, L, U = linalg.lu(A)

# Solve Linear System of Equations:
LU, P = linalg.lu_factor(A)
x = linalg.lu_solve( (LU, P), b)
```

Upper- and lower-trangular matrix:

```
L = linalg.triu(LU, -1) + np.identity(3)
U = linalg.triu(LU)
```

```
Q, R = linalg.qr(A)
```